

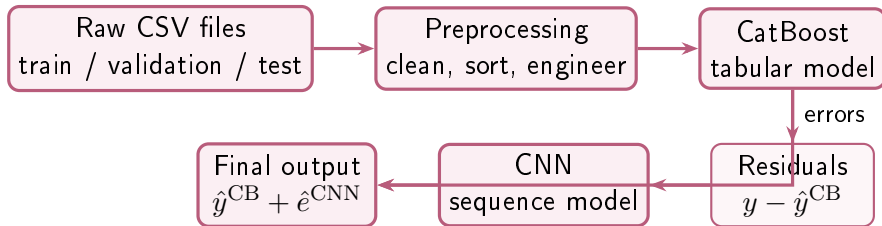
IML Project – Financial Prediction

CatBoost + CNN residual learning for monthly return prediction

Tanush Seal Aarush Ram Anandh

- Target: monthly gross return $y_{j,t}$.
- Inputs: engineered tabular features $x_{j,t}$ and causal sequences $\mathcal{S}_{j,t}$.
- Predictor: $\hat{y}_{j,t} = \hat{y}_{j,t}^{\text{CB}} + \hat{e}_{j,t}^{\text{CNN}}$.
- Data structure: monthly panel indexed by company j and time t .

Pipeline Diagram



- The top path learns the main tabular pattern.
- The bottom path learns the leftover time pattern.

What This Project Does

- The goal is to estimate $y_{j,t}$ for each company-month pair (j, t) .
- The data form a monthly panel $\{(j, t, r_{j,t})\}$.
- The model is split into two parts:
 - CatBoost learns $f_{CB}(x_{j,t})$.
 - CNN learns the residual $e_{j,t} = y_{j,t} - \hat{y}_{j,t}^{CB}$.
- The pipeline preserves chronology and prevents leakage: $\tau \leq t$ only.

Big Picture Flow

- 1 Load $\mathcal{D}_{\text{train}}$, $\mathcal{D}_{\text{val}}$, and $\mathcal{D}_{\text{test}}$.
- 2 Build $x_{j,t} = \phi(\{r_{j,\tau} : \tau \leq t\})$.
- 3 Fit $\hat{f}_{\text{CB}}$ on tabular features.
- 4 Compute $e_{j,t} = y_{j,t} - \hat{y}_{j,t}^{\text{CB}}$.
- 5 Fit $\hat{f}_{\text{CNN}}$ on sequence tensors $\mathcal{S}_{j,t}$.
- 6 Predict $\hat{y}_{j,t} = \hat{y}_{j,t}^{\text{CB}} + \hat{e}_{j,t}^{\text{CNN}}$.
- 7 Store metrics, predictions, and HTML logs.

Why Use Two Models?

- CatBoost models $x_{j,t} \in \mathbb{R}^d$.
- It handles numerical and categorical structure well.
- Some signal is temporal and lives in the residuals.
- CNN models short windows $\mathcal{S}_{j,t} \in \mathbb{R}^{d \times L}$.
- The final prediction is the sum of both parts:

$$\hat{y}_{j,t} = \hat{y}_{j,t}^{\text{CB}} + \hat{y}_{j,t}^{\text{CNN}}.$$

Simple View of the Math

Let:

- $y_{j,t}$ be the true return for company j at month t ,
- $x_{j,t}$ be the feature vector for that row,
- $\mathcal{S}_{j,t}$ be the short sequence used by the CNN.

The model works like this:

$$y_{j,t} \approx f_{\text{CB}}(x_{j,t}) + f_{\text{CNN}}(\mathcal{S}_{j,t}).$$

In plain language:

- CatBoost explains the obvious part.
- CNN explains the leftover part.

How The Data Is Processed

- The raw row for company j at month t is called $r_{j,t}$.
- Preprocessing creates a cleaned feature row $x_{j,t}$.
- The cleaned row depends only on current and past months:

$$x_{j,t} = \phi(\{r_{j,\tau} : \tau \leq t\}).$$

- This keeps the pipeline leakage-safe and causal.
- Lags and rolling features use $t-1, t-3, t-6$ style history.

Feature Construction Math

Lag features

For lag $\ell \in \{1, 3, 6\}$, the lag feature is the earlier target value:

$$\text{lag_ret}_\ell(j, t) = y_{j, t-\ell}.$$

Rolling features

A rolling mean over window w is computed from past values only:

$$\text{rolling_mean}_w(j, t) = \frac{1}{|I_{j,t}^{(w)}|} \sum_{\tau \in I_{j,t}^{(w)}} y_{j,\tau}.$$

The same idea is used for rolling standard deviation.

CatBoost in Plain Language

- CatBoost is a boosting model made of many small decision trees.
- Each new tree tries to fix the mistakes of the previous trees.
- After many steps, the model becomes a strong predictor.
- It works well when features are a mix of numeric values and categories.

A simple training objective is:

$$\mathcal{L}_{\text{CB}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

How Boosting Works

- Start with a simple guess, usually the average target.
- Measure the error made by that guess.
- Train a small tree to predict the error.
- Add the tree to the current model.
- Repeat many times until the error is low enough.

In symbol form:

$$F_M(x) = F_0(x) + \sum_{m=1}^M \eta_m h_m(x),$$

where each h_m is a small tree and η_m is the step size.

Preprocessing Pipeline

- Load $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}}$.
- Concatenate temporarily: $\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}} \cup \mathcal{D}_{\text{test}}$.
- Sort by (co_code, Month).
- Collapse duplicate keys (j, t) .
- Coerce Month, co_code, and $y_{j,t}$.
- Build lag_ret_ℓ , rolling_mean_w , and missing flags.
- Recover the original split sets.

Implementation Details Used In Code

- Duplicate company-month rows are collapsed by keeping the last observation.
- Missing categorical values are filled with `__MISSING__`.
- Lag steps are set to 1, 3, and 6 months.
- Rolling windows are set to 3 and 6 months.
- CNN sequences use a window length of 6.
- CatBoost predictions are stored as `cb_pred`.
- Residual targets are stored as `base_residual` and `residual_target`.

Why Preprocessing Matters

- Financial panel data are time ordered.
- We must not use future months when building features for past rows.
- Lag features use only earlier months.
- Rolling features also look only backward in time.
- Missing-value flags help the model learn when data are absent.

CatBoost Training in This Project

- Train CatBoost only on $\mathcal{D}_{\text{train}}$.
- Use $\mathcal{D}_{\text{val}}$ for model checking and early stopping.
- Pass categorical features directly into the Pool.
- Predict on $\mathcal{D}_{\text{val}}$ and $\mathcal{D}_{\text{test}}$.
- Compute residuals:

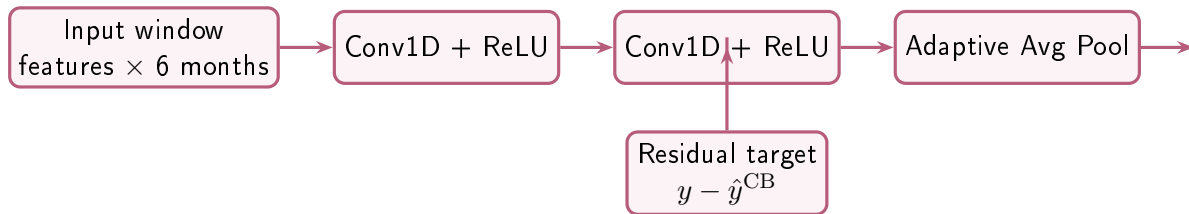
$$e_{j,t} = y_{j,t} - \hat{y}_{j,t}^{\text{CB}}.$$

CNN Training in This Project

- CNN uses a window $\mathcal{S}_{j,t} \in \mathbb{R}^{d \times L}$.
- Each window is standardized with training statistics (μ, σ) .
- The CNN estimates $\hat{e}_{j,t}^{\text{CNN}}$, not $y_{j,t}$ directly.
- The loss is mean squared error on residuals:

$$\mathcal{L}_{\text{CNN}} = \frac{1}{m} \sum_{i=1}^m (e_i - \hat{e}_i)^2.$$

CNN Block Diagram



- The CNN sees a window of past rows, not a single row.
- The output is one correction value for the target return.

How the CNN Sees the Data

- Input shape is roughly: features $\times$ time window.
- Convolution layers scan across the time axis.
- This helps the model detect local patterns such as short momentum or reversal effects.
- After convolution, the model uses pooling and a linear output layer.

- At inference time, append new rows to the historical panel.
- Rebuild features with the same map ϕ .
- Predict $\hat{y}_{j,t}^{\text{CB}}$ first.
- Build a valid window $\mathcal{S}_{j,t}$ if $|H_{j,t}| \geq L$.
- Add the CNN correction and write $\hat{y}_{j,t}$.
- Save outputs, metrics, and HTML summaries.

What Gets Logged

- Final predictions $\hat{y}_{j,t}$ for each row.
- Base predictions $\hat{y}_{j,t}^{\text{CB}}$ and corrections $\hat{e}_{j,t}^{\text{CNN}}$.
- Metrics such as RMSE, MAE, R^2 , and directional accuracy.
- Learning curves for CatBoost and CNN.
- Error distributions and monthly summaries.

Takeaway

- CatBoost handles the main tabular structure.
- CNN handles the leftover time pattern in the residuals.
- Preprocessing protects against leakage.
- Inference repeats the same logic used during training.
- Logging makes the model easier to inspect and compare.